

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Sakkrobot fejlesztése és adaptálása új bábuk kezelésére

Szakdolgozat

Készítette
Székely Máté Zsombor

Konzulens
Szekeres Kornél

2026. október 29.

Tartalomjegyzék

Kivonat i

Abstract ii

1. Bevezetés 1

2. Sakkszabályok és tündérsakk 2
2.1. A FIDE-szabályok motor szempontjából fontos részei . . . 2
2.2. A tündérsakk alapelvei . . . 2

3. Előzmények és szakirodalmi háttér 4
3.1. Táblareprezentáció . . . 4
3.2. Keresés . . . 4
3.3. Állásértékelés: kézzel írt függvénytől az NNUE-ig . . . 5
3.4. UCI protokoll . . . 5
3.5. Hasonló alkotások . . . 5

4. Szoftverarchitektúra és UCI kommunikációs réteg 6
4.1. Áttekintés . . . 6
4.2. Rétegek és felelősségek . . . 6
4.3. Szálkezelés . . . 7
4.4. Verziókezelés és mérési infrastruktúra . . . 7
4.5. A UCI kommunikációs réteg . . . 8

5. Táblareprezentáció és lépésgenerálás 9
5.1. Az állás tárolása . . . 9
5.2. A lépés kódolása . . . 9
5.3. Támadási táblák és mágikus bitboardok . . . 9
5.4. Pszeudolegális generálás és legalitásvizsgálat . . . 10
5.5. Döntetlenek és ismétlésfigyelés . . . 10

6. A keresőmotor 11
6.1. A keresés kerete . . . 11
6.2. Transzpozíciós tábla (v3, klaszterek v13, zármentes v23) . . . 12
6.3. Lépésrendezés . . . 12
6.4. Vágások, redukciók és kiterjesztések . . . 13
6.5. Nyugalmi keresés (v1) . . . 13
6.6. Állásértékelés . . . 14
6.7. Időkezelés . . . 14
6.8. Többszálú keresés: Lazy SMP (v16) . . . 14
6.9. Nehézségi szintek . . . 15

7. A grafikus felhasználói felület **16**
7.1. Felépítés . . . 16
7.2. A főablak . . . 17
7.3. Interakció a táblán . . . 17
7.4. Beállítások . . . 18
7.5. Játék sakkórával és robot-robot mód . . . 19

8. Tündérsakk-kiterjesztés: az Amazon sakk **21**
8.1. A változat szabályai . . . 21
8.2. Megvalósítás a motorban . . . 21
8.3. Ellenőrzés . . . 22
8.4. Hogyan játszik a kereső Amazonnal? . . . 22

9. Mérések és értékelés **24**
9.1. A lépésgenerálás helyessége és sebessége . . . 24
9.2. A döntéshozatal hatékonysága . . . 24
9.3. A kereső fejlődése verzióról verzióra . . . 25
9.4. Abszolút erősség: mérés a Stockfish ellen . . . 26
9.5. Mérések a konzolos backendben . . . 27

10. Összefoglalás és továbbfejlesztési lehetőségek **28**
10.1. Az elvégzett munka értékelése . . . 28
10.2. Továbbfejlesztési lehetőségek . . . 28

Irodalomjegyzék **30**

Függelék **32**
F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról . . . 32

HALLGATÓI NYILATKOZAT

Alulírott *Székely Máté Zsombor*, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2026. október 29.

Székely Máté Zsombor
hallgató

Kivonat

A szakdolgozatban egy saját fejlesztésű, C++20 nyelvű sakkmotort mutatok be, amely a klasszikus FIDE-szabályok mellett tündérsakk-bábuk kezelésére is képes. A motor bitboard-alapú táblareprezentációt és mágikus bitboardokkal gyorsított lépésgenerátort használ, döntéseit alfa-béta alapú kereséssel hozza, amelyet transzpozíciós tábla, PVS, aspirációs ablak, többféle vágási, redukciós és kiterjesztési technika, többszálú (Lazy SMP) keresés, valamint NNUE neurális hálós állásértékelés egészít ki. A sakkprogram HalfKP (régebbi verzió) mellett HalfKAv2 hálót is támogat. A motor szabványos UCI protokollon kommunikál, és saját, Qt 6 alapú grafikus felülettel rendelkezik, amely négy nehézségi szintet, sakkórát, lépésvisszavonást és játszma-visszajátszást is kínál.

A kereső fejlődését verziókezelt, robot-robot mérésekkel követtem: húsz mérés, összesen több mint tizenkétezer játszma igazolja az egyes optimalizációk hatását. A lépésgenerálás helyességét öt szabványos perft-tesztállás igazolja, egy szálon másodpercenként 25–33 millió állással. A korlátozott erősségű Stockfish 17.1 elleni 300 játszma alapján a motor ereje a CCRL blitz skálán kb. 3100 Elo. A tündérsakk-kiterjesztést az Amazon (vezér + huszár) bábun keresztül valósítottam meg: a bábu a lépésgenerátortól az értékelésen át a felületig minden réteget érint, mégis kevés, jól körülhatárolt módosítással illeszkedett a meglévő architektúrába. A dolgozat végül önjátszmák alapján elemzi, hogyan használja a kereső az új bábut.

Abstract

This thesis presents a chess engine written from scratch in C++20 that plays by the classical FIDE rules and can also handle fairy chess pieces. The engine uses a bitboard board representation with a magic-bitboard move generator. Its decisions come from an alpha-beta search extended with a transposition table, principal variation search, aspiration windows, a range of pruning, reduction and extension techniques, multi-threaded (Lazy SMP) search and NNUE neural network evaluation, supporting both HalfKP (older version) and HalfKAv2 network. The engine communicates through the standard UCI protocol and has its own Qt 6 graphical user interface with four difficulty levels, chess clocks, move takeback and game review.

The development of the search was tracked with versioned bot-versus-bot measurements: twenty matches with more than twelve thousand games quantify the effect of each optimization. The correctness of the move generator is verified on five standard perft positions, running at 25–33 million positions per second on a single thread. Based on 300 games against strength-limited Stockfish 17.1, the engine plays at about 3100 Elo on the CCRL blitz scale. The fairy chess extension was implemented through the Amazon (queen + knight) piece. Although the piece affects every layer from move generation through evaluation to the user interface, it fit into the existing architecture with a small number of well-contained changes. Finally, the thesis analyses self-play games to show how the search makes use of the new piece.

1. fejezet

Bevezetés

A sakk évtizedek óta a mesterséges intelligencia kutatásának egyik kedvelt terepe: szabályai pontosan definiáltak, állapottere azonban olyan hatalmas, hogy a teljes játékfa bejárása gyakorlatilag lehetetlen. Claude Shannon már 1950-ben megfogalmazta a számítógépes sakk alapproblémáját, és megbecsülte, hogy egy átlagos játszma lehetséges folytatásainak száma nagyságrendileg 10^{120} [17]. Egy erős sakkmotor ezért nem brute-force kereséssel, hanem jól megválasztott vágási és értékelési heurisztikákkal dolgozik.

A szakdolgozatom célja egy saját fejlesztésű sakkmotor megtervezése és megvalósítása volt, amely

- a klasszikus (FIDE) szabályok szerint játszik,
- az iparági szabványnak tekinthető UCI (Universal Chess Interface) protokollon keresztül más programokkal is együttműködik,
- saját, interaktív grafikus felhasználói felülettel rendelkezik,
- és úgy épül fel, hogy új, nem szabványos (tündérsakk) bábukkal is bővíthető legyen.

A tündérsakk-kiterjesztés bemutatására az Amazon bábut választottam, amely a vezér és a huszár lépésmódját egyesíti. Egy ilyen bábu bevezetése nemcsak a lépésgenerátort érinti, hanem a cserék kiértékelését, a lépésrendezést és az állásértékelést is, így jól megmutatja, mennyire általános egy motor architektúrája.

A munka két forráskód-tárolóban készült. A backend a motor konzolos változata, ebben fut a UCI és a robot-robot elleni tömeges mérés. A GUI projekt ugyanezt a motort egy grafikus felülettel egészíti ki. A fejlesztés 2025 decemberében indult, a motor 2026 áprilisától kezdve mért, verziózott lépésekben fejlődött: minden módosított keresőt egy korábbi verzió ellen több ezer játszmában teszteltem, és csak a bizonyítottan erősebb változatot tartottam meg.

A dolgozat felépítése

A 2. fejezet bemutatja a vonatkozó FIDE-szabályokat és a tündérsakk bábuk alapelveit. A 3. fejezet a sakkmotorok szakirodalmi hátterét foglalja össze. A 4. fejezet a szoftverarchitektúrát és a UCI kommunikációs réteget, az 5. fejezet a táblareprezentációt és a lépésgenerálást ismerteti. A 6. fejezet a keresőmotor összes funkcióját és optimalizációját tárgyalja, verziók szerint. A 7. fejezet a grafikus felületet, a 8. fejezet az Amazon bábuval bővített játékváltozatot mutatja be. A 9. fejezetben mérésekkel igazolom a lépésgenerálás helyességét és hatékonyságát, valamint a motor fejlődését. A dolgozatot az eredmények értékelése és a továbbfejlesztési lehetőségek zárják.

2. fejezet

Sakkszabályok és tündérsakk

2.1. A FIDE-szabályok motor szempontjából fontos részei

A motor a Nemzetközi Sakkszövetség (FIDE) hivatalos szabályzatát [11] valósítja meg. A játék 8×8-as táblán, két fél között zajlik. Mindkét fél 16 bábuval indul: király, vezér, két bástya, két futó, két huszár és nyolc gyalog. A szabályok közül a következők igényelnek a lépésgenerátorban külön kezelést:

- *Csúszó bábuk:* a bástya vízszintesen és függőlegesen, a futó átlósan, a vezér mindkét módon léphet tetszőleges távolságra, de nem ugorhat át más bábut.
- *Ugró bábu:* a huszár „L" alakban lép, és átugorhatja a köztes mezőket.
- *Gyalog:* előre egyet, kiinduló sorából kettőt léphet, átlósan üt. Ha a hetedik (sötét esetén a második) sorról az utolsó sorra lép, átváltozik huszárrá, futóvá, bástyává vagy vezérré.
- *En passant:* ha a gyalog kettőt lép, és ezzel egy ellenfél gyalog mellé érkezik, az ellenfél a következő lépésben úgy ütheti, mintha csak egyet lépett volna.
- *Sáncolás:* a király két mezőt lép a bástya felé, a bástya átugrik mellé. Feltétele, hogy egyik bábu se lépett még, köztük ne álljon bábu, és a király se sakkban ne álljon, se megtámadott mezőn ne haladjon át, se ilyenre ne érkezzen.
- *Sakk, matt, patt:* szabálytalan minden lépés, amely után a lépő fél királya támadás alatt marad. Ha a lépésre következő félnek nincs szabályos lépése, a játszma sakkban matttal, egyébként pattal (döntetlen) ér véget.
- *Döntetlen szabályok:* a háromszori állásismétlés, az ötvenlépéses szabály (50 lépés ütés és gyaloglépés nélkül), valamint az elégtelen anyag (pl. király és futó a király ellen), amellyel matt nem adható.

A versenyjátszmákban a gondolkodási időt sakkóra méri. A motor a két legelterjedtebb módot támogatja: a fix lépésenkénti időt és az időbeosztást, ahol mindkét fél alapidővel indul, és minden lépés után bónuszidőt kap.

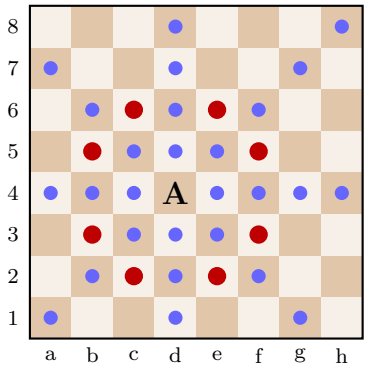
2.2. A tündérsakk alapelvei

A tündérsakk gyűjtőfogalom: minden olyan változatot ide sorolunk, amely nem szabványos bábukat, táblát vagy szabályokat használ [15]. A tündérbábuk mozgását általában a hagyományos bábuk alapmozgásaiból, más néven atomokból rakják össze. Ralph Betza

jelölésrendszerében [1] például `W` a vízszintes/függőleges egylépéses, `F` az átlós egylépéses, `N` a huszárugrás; a betű megduplázása (`WW`, `FF`) vagy a `R`, `B` jelek a csúszó, tetszőleges távolságú változatot jelölik. Ebben a rendszerben a vezér `Q = RB`, a huszár `N`.

Az összetett bábuk két vagy több atom unióját mozgatják. A legismertebbek:

- Arkangyal (BN): futó + huszár,
- Kancellár (RN): bástya + huszár,
- Amazon (QN): vezér + huszár.



2.1. ábra. Az Amazon lépései a d4 mezőről: kék a vezérszerű , piros a huszárszerű célmezők.

Az Amazon (2.1. ábra) üres táblán egy középső mezőről akár 35 mezőt is elérhet (27 vezérszerű + 8 huszárugrás), szemben a vezér 27 mezőjével. A huszárugrás az egyetlen olyan támadási forma, amelyet a vezér nem tud kivédeni közbeállással, ezért az Amazon egyedül is mattot adhat a magára hagyott királynak. Mivel a király ilyen bábu ellen különösen sebezhető, az Amazon jelentősen felülmúlja a vezért; a szakirodalom értékét általában 12–13 gyalogegységre teszi [19]. A motorban az Amazon a vezér helyén áll, és az ilyen változatban a vezér teljesen eltűnik a játékból: a gyalog is Amazonná változik a vezér helyett.

3. fejezet

Előzmények és szakirodalmi háttér

3.1. Táblareprezentáció

A modern sakkmotorok túlnyomó többsége bitboardokkal írja le az állást [3]. A bitboard egy 64 bites egész szám, amelynek i-edik bitje azt jelzi, hogy az i-edik mezőn áll-e az adott típusú bábu. Bábutípusonként és színenként egy-egy bitboardot tárolva a lépésgenerálás és a támadásszámítás nagy része néhány bitművelettel (ÉS, VAGY, eltolás, `popcount`, legalacsonyabb bit keresése) elvégezhető, amelyeket a modern processzorok egyetlen utasítással hajtanak végre.

A csúszó bábuk (bástya, futó, vezér) támadásai a köztes mezők foglaltságától függenek. Erre a legelterjedtebb megoldás a magic bitboard technika [5]: a releváns foglaltsági bitek egy előre megkeresett magic konstanssal szorozva és eltolva egy tömbindexet adnak, így a támadási halmaz egyetlen táblázat-kiolvasással megkapható.

Az állások gyors azonosítására a Zobrist-hash [24] szolgál: minden (bábu, mező) párhoz, a lépésjoghoz, a sáncjogokhoz és az en passant oszlophoz egy-egy véletlen 64 bites számot rendelünk, és az állás kulcsa ezek XOR-ja. Lépéskor a kulcs inkrementálisan, néhány XOR-ral frissíthető. A kulcs az ismétlésfigyeléshez és a transzpozíciós táblához is használható.

A lépésgenerátor helyességét a perft (performance test) módszerrel szokás ellenőrizni: adott mélységig megszámoljuk az összes szabályos lépéssorozat végállását, és az eredményt ismert referenciaértékekkel vetjük össze [6].

3.2. Keresés

A kétszemélyes játékok klasszikus keresőeljárása a minimax, illetve ennek negamax alakja (amennyiben a két félnek a szabályai szimmetrikusak). Az alfa-béta vágás [12] ugyanazt az eredményt adja, mint a minimax, de jó lépésrendezés esetén a vizsgált csomópontok száma nagyjából a négyzetgyökére csökken. Erre az alapra épülnek a modern motorok további technikái:

- *Iteratív mélyítés:* a motor egyre nagyobb mélységgel keres, és az előző iteráció eredményét a lépésrendezéshez és az időkezeléshez használja.
- *Transzpozíciós tábla (TT):* a már kiértékelt állások eredményét Zobrist-kulccsal indexelt hash-táblában tárolja [9].
- *Principal Variation Search (PVS):* a legjobbnak vélt lépés után a többit csak nulla szélességű ablakkal ellenőrzi.

- *Nyugalmi keresés* (*quiescence search*): a horizont-effektus elkerülésére a levélcsomópontokban az ütéseket tovább vizsgálja.
- *Vágások és redukciók:* Null-Move pruning, Late Move reduction (LMR), futility, razoring jellegű vágások [7].
- *Lépésrendezés:* TT-move, MVV-LVA, Static Exchange Evaluation (SEE), killer move, move history, counter move history.
- *Párhuzamosítás:* a Lazy SMP eljárásban több szál keresi ugyanazt a fát, közös transzpozíciós táblán keresztül osztva meg az eredményeket [4].

3.3. Állásértékelés: kézzel írt függvénytől az NNUE-ig

Hagyományosan az állásértékelő függvény kézzel hangolt tagok összege: anyagérték, bábumező táblák, mobilitás, gyalogszerkezet, királybiztonság stb. 2018-ban Yu Nasu bemutatta az NNUE (Efficiently Updatable Neural Network) architektúrát [14], amelyet 2020-ban a Stockfish is átvett. Az NNUE első rétege nagyon széles, de ritka bemenetű: a bemeneti jellemzők (például „saját király a g1-en, ellenséges huszár f6-on") egy lépés hatására csak néhány helyen változnak, így az első réteg kimenete (az akkumulátor) lépésenként inkrementálisan frissíthető. Ettől lesz az értékelés elég gyors ahhoz, hogy a keresés minden csomópontjában használható legyen.

A motor két hálót is használ: a régebbi *HalfKP* formátumú Stockfish-hálót a nyílt forrású `nnue-probe` könyvtáron [18] keresztül, valamint egy újabb, *HalfKAv2_hm* formátumú hálót [22].

3.4. UCI protokoll

A UCI [13] szöveges, soronkénti protokoll, amelyen keresztül egy grafikus felület (pl. Arena, Cute Chess, Lichess-bot) a motor standard bemenetére és kimenetére írva vezérli a motort. A legfontosabb parancsok: `uci` (azonosítás, válasz: `uciok`), `isready` (`readyok`), `ucinewgame`, `position startpos|fen … moves …`, `go` (paraméterei: `wtime`, `btime`, `winc`, `binc`, `movetime`), `stop` és `quit`. A motor a keresés végén `bestmove` sorral válaszol.

3.5. Hasonló alkotások

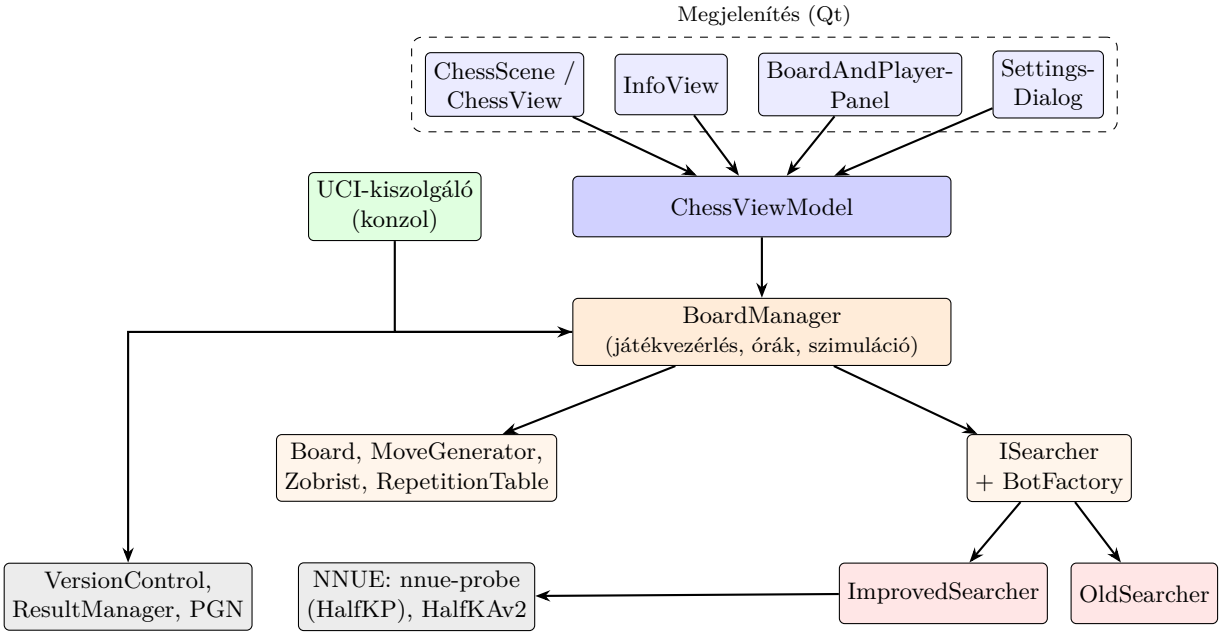
A legerősebb nyílt forrású motor a Stockfish [23], amely a NNUE-t a fent vázolt keresési technikák igen kifinomult változataival kombinálja. Tündérsakk-változatok tekintetében a Fairy-Stockfish [10] a meghatározó: ez a Stockfish olyan elágazása, amely konfigurációs fájlból tetszőleges bábukat és szabályokat képes betölteni. Ennek a motor célja egy áttekinthető, mérésekkel alátámasztott, saját kódbázis volt, amelybe a tündérbábuk támogatása kis, jól körülhatárolt módosításokkal illeszthető.

4. fejezet

Szoftverarchitektúra és UCI kommunikációs réteg

4.1. Áttekintés

A rendszer C++20 nyelven készült, és két tárolóban él. A backend egy konzolos projekt, amely a motort, a UCI-kiszolgálót és a robot-robot elleni mérőkörnyezetet tartalmazza. A GUI projekt ugyanazokat a motorforrásokat egy grafikus felülettel egészíti ki. A két tároló motorrésze forrásszinten megegyezik, így minden kereső- és lépésgenerátor-fejlesztés mindkét programban elérhető. A rétegeket a 4.1. ábra mutatja.



4.1. ábra. A rendszer rétegei. A kék elemek csak a GUI projektben, a zöld csak a konzolos backendben léteznek, a többi közös.

4.2. Rétegek és felelősségek

- **Szabályréteg** (`Board`, `MoveGenerator`, `Zobrist`, `RepetitionTable`): az állás tárolása, a pszeudolegális lépések előállítása, a lépés végrehajtása és visszavonása, a

sakk-, matt-, patt- és döntetlenfigyelés. Ez a réteg semmit sem tud a keresésről és a felületről.

- **Játékvezérlő** (`BoardManager`): egy játszma teljes állapotát kezeli: a sakkórákat, a robotok létrehozását és időbeállításait, a lépések visszavonását, a játszma végének felismerését, a megnyitáskönyvből indított robot-robot szimulációt és a UCI-hurkot.
- **Keresők** (`ISearcher` interfész): minden robot ugyanazt az interfészt valósítja meg (`setState`, `GetRobotMove`, `setDifficulty`, időkezelő függvények, `stopSearch`). A `BotFactory` típus alapján példányosítja az aktuális (`ImprovedSearcher`) vagy a referenciaként megtartott korábbi (`OldSearcher`) keresőt. A két kereső külön névtérben él, így egyszerre, egymás ellen is futtathatók.
- **Segédmodulok**: a `VersionControl` a keresők forráskódjáról SHA-256 lenyomatot készít, és minden új kódállapotot `vN` néven archivál; a `ResultManager` a robot-robot játszmák eredményét és PGN-jét verziópáronként menti; a `PGNFormatter` a lépéseket szabványos algebrai jelölésre (SAN) alakítja.
- **Megjelenítés** (csak GUI): MVVM jellegű felépítés, ahol a `ChessViewModel` közvetít a Qt-nézetek és a `BoardManager` között (részletesen a 7. fejezet).

A beállításokat egyszerű adatszerkezetek (`RobotSettings`, `BoardSettings`, `TimeSettings`, `VariantSettings`) hordozzák; ezek `operator!=` műveletével a nézetmodell pontosan eldönti, hogy egy változtatás új játszmát igényel-e, vagy elég a meglévőt frissíteni.

4.3. Szálkezelés

A keresés mindig külön szálon fut, hogy a felület és a sakkóra ne akadjon meg. A GUI a robot lépését QThread-ben számolja, és az eredményt sorba állított jelzéssel juttatja vissza a fő szálra; egy keresésazonosító biztosítja, hogy egy időközben megszakított (pl. visszavont) keresés eredménye ne kerüljön a táblára. Maga a kereső további segédszálakat indíthat (Lazy SMP, a 6.8. szakasz). A tömeges robot-robot szimuláció során minden szál saját `BoardManager` példánnyal és saját robotokkal dolgozik, csak a statisztika összesítése és a konzolkimenet közös, mutex-szel védve.

4.4. Verziókezelés és mérési infrastruktúra

A fejlesztés egyik legfontosabb eszköze az automatikus verziózás volt. Robot-robot szimuláció indításakor a `VersionControl` kiszámítja mindkét kereső forráskönyvtárának hash-ét, ha ilyen kódállapot még nem volt, a forrásokat `generatedBotVersions/<kereső>/vN/src` alá másolja. Az eredmények `botvsbotresults/<kereső1>-<kereső2>/<kereső1_vN>-<kereső2_vM>/` mappába kerülnek: a `results.txt` az összesített győzelem/döntetlen/vereség számokat, a `Matches` alkönyvtár pedig eredmény szerint csoportosítva minden játszma PGN-jét tartalmazza. Így minden mérés utólag is visszakereshető és megismételhető.

A szimuláció a 22 234 különböző állást tartalmazó megnyitáskönyvből (assets/openings.txt) véletlenszerű kezdőállást választ, és minden kezdőállásból két játszmát játszat, színcserével. Ez kiszűri a megnyitásválasztásból és a világos kezdés előnyéből adódó torzítást.

4.5. A UCI kommunikációs réteg

A konzolos program UCI-kiszolgálóként indul (`BoardManager::runUCIService`). A szöveges parancsokat soronként olvassa, és az első szó alapján dolgozza fel; a lépések UCI-formátumú (pl. `e7e8q`) szövegét a `UCIParsing` alakítja belső lépéssé és vissza. A `go` parancs fix idős (`movetime`) és órás (`wtime`/`btime`/`winc`/`binc`) módot is ismer; ez utóbbit a robot időkezelője (a 6.7. szakasz) használja fel. Egy rövid, valódi munkamenet:

```
> uci                                   < id name <bot's name>
                                        < id author <name>
                                        < uciok
> isready                               < readyok
> position startpos moves e2e4 e7e5
> go movetime 1000                      < bestmove g1f3
> position fen 6k1/5ppp/8/8/8/8/5PPP/3Q2K1 w - - 0 1
> go movetime 500                       < bestmove d1d8
```

4.1. lista. UCI munkamenet (bemenet és a motor válaszai)

A protokollnak köszönhetően a motor bármely UCI-kompatibilis felületről (pl. Cute Chess, általam haszsnált környezet) használható, és más motorok ellen is versenyeztethető.

5. fejezet

Táblareprezentáció és lépésgenerálás

5.1. Az állás tárolása

A `Board` osztály az állást színenként és bábutípusonként egy-egy 64 bites bitboardban tárolja (m_bitboards[2][PIECE_TYPE_COUNT]); a mezők számozása a1 = 0-tól h8 = 63-ig halad. Ezek mellett színenként egy foglaltsági bitboard és egy összesített foglaltság is rendelkezésre áll, amelyeket minden lépés inkrementálisan frissít. A nem bitboard jellegű állapotot (sáncjogok, en passant mező, ütött bábu típusa, félléptetés-számláló, lépésszám, Zobrist-kulcs) egy kompakt `BoardState` rekord tárolja, és ezekből egy 1024 elemű tömb őrzi a teljes előzményt. Így a lépés visszavonásához nem kell semmit újraszámolni, csak az előző rekordra visszalépni.

5.2. A lépés kódolása

Egy lépés (`Move`) 16 bitbe tömörítve tárolja a kiinduló mezőt (6 bit), a célmezőt (6 bit) és egy 4 bites jelzőt, mellette egy bájton a mozgó bábu típusát. A jelzőbitek külön-külön kódolják az ütést és az átváltozást, a maradék két bit pedig a lépés altípusát: dupla gyaloglépés, rövid vagy hosszú sánc, en passant, illetve átváltozásnál a választott bábut (huszár, futó, bástya, vezér). A tömör forma miatt egy lépés egyetlen gépi szóban mozgatható, és a transzpozíciós tábla bejegyzésébe is belefér. A lépéslista (`MoveList`) egy 256 elemű, rögzített méretű tömb: a keresés során így egyáltalán nincs dinamikus memóriafoglalás.

5.3. Támadási táblák és mágikus bitboardok

A program indulásakor egyszer (`std::call_once`) épülnek fel az előre kiszámított táblák:

- a huszár, a király és a gyalogok (színenként) támadási halmaza mind a 64 mezőre,
- a bástyák (64×4096) és futók (64×512) mágikus bitboard-táblái, előre megkeresett mágikus konstansokkal és eltolásokkal,
- a szabad gyalog maszkok, amelyeket a kézzel írt értékelés és a keresés is használ.

Egy csúszó bábu támadásainak lekérdezése így egy maszkolás, egy szorzás, egy eltolás és egy táblaolvasás; a vezér támadása a bástya- és a futótámadás uniója.

5.4. Pszeudolegális generálás és legalitásvizsgálat

A `MoveGenerator` a lépő fél minden bábujára előállítja a pszeudolegális lépéseket, vagyis azokat, amelyek a bábu mozgásszabályainak megfelelnek, de esetleg sakkban hagyják a saját királyt. A sáncolásnál a generátor már itt ellenőrzi, hogy a király nem áll sakkban, és nem halad át megtámadott mezőn. A nyugalmi kereséshez létezik egy csak ütéseket előállító mód is.

A legalitás eldöntése a `MakeMove` feladata: a lépés végrehajtása után megvizsgálja, hogy a lépő fél királyát támadja-e valami (isSquareAttacked), és ha igen, a lépést azonnal visszavonja és elutasítja. A támadásvizsgálat fordított irányban dolgozik: a király mezőjéről indítja a huszár-, király-, gyalog- és csúszótámadásokat, és ezek metszetét képezi az ellenfél megfelelő bitboardjaival. Ez a megközelítés egyszerűbb a teljesen legális generálásnál, és a keresésben nem drágább, mert a lépések jelentős része a vágások miatt végre sem hajtódik.

A lépés végrehajtásakor a sáncjogok frissítéséhez egy mezőnkénti maszk (castling_mask) elég: ha egy lépés a király vagy egy bástya kiinduló mezőjét érinti, a megfelelő jog egyetlen ÉS művelettel törlődik. A Zobrist-kulcsot a `MakeMove` inkrementálisan, XOR-okkal frissíti, a GenerateFullHash csak a FEN betöltésekor számol teljes kulcsot, mivel ez egy lassú művelet. A `MakeMove` kétféle módban hívható: keresési módban csak a fenti minimális munkát végzi, játszma módban (a felület és a játékvezérlő számára) eltárolja a lépést, a táblaképet, a sakkjelzést és a következő állás szabályos lépéseit is, amelyekből a lépéslista, a SAN-jelölés és a visszajátszás dolgozik.

5.5. Döntetlenek és ismétlésfigyelés

A `RepetitionTable` a játszma és a keresés során előforduló állások kulcsait tárolja. Mivel ütés vagy gyaloglépés után korábbi állás már nem ismétlődhet meg, minden ilyen visszafordíthatatlan lépés új ablakot nyit, és az ismétlést csak az aktuális ablakban kell keresni. A játszmában a háromszori ismétlés a döntetlen, a keresésben viszont már a kétszeri is annak számít: ha egy állás egyszer már előfordult, a motor az ismétlésre vezető ágat döntetlenként értékeli, így felesleges mélységet takarít meg. Ezen felül a tábla az ötvenlépéses szabályt (100 féllépés) és az elégtelen anyagot is felismeri (csak királyok, király és egy könnyűtiszt, azonos színű futók).

A lépésgenerátor helyességét és sebességét a 9.1. szakasz perft-mérései igazolják.

6. fejezet

A keresőmotor

A motor agya az `ImprovedSearcher` névtér `Searcher` osztálya. Ez a fejezet a kereső minden funkcióját bemutatja. A zárójelben szereplő **vN** jelzés azt a verziót adja meg, amelyben az adott technika először megjelent, a hozzá tartozó mérést a 9.3. szakasz tartalmazza. A kereső a v1-ben szándékosan egy minimális magról (alfa-béta, NNUE, nyugalmi keresés) indult, és minden további elem külön verzióként, mérve került bele.

6.1. A keresés kerete

Iteratív mélyítés (v1). A kereső 1-es mélységtől kezdve egyre mélyebben keres, amíg az időkezelő meg nem állítja. Minden befejezett iteráció legjobb lépése és értéke megmarad, így egy félbeszakított iteráció sem okoz rossz döntést, az előző iteráció eredménye pedig a következő lépésrendezését és ablakát segíti.

Negamax alfa-béta, fail-soft (v4, teljesen v23). A rekurzív negamax a szokásos alfa-béta vágással dolgozik. A fail-soft változat az ablakon kívül eső értékeket is a lehető legpontosabban adja vissza, ami a transzpozíciós tábla bejegyzéseit és az aspirációs ablak újrakeresését is pontosabbá teszi.

Principal Variation Search (v7). Egy csomópont első (legjobbnak vélt) lépését teljes ablakkal, a többit nulla szélességű ablakkal ($[\alpha, \alpha + 1]$) keresi. Ha egy későbbi lépés mégis jobbnak bizonyul, teljes ablakkal újrakeresi. Jó lépésrendezés esetén a nulla ablakos keresések sokkal olcsóbbak.

Aspirációs ablak (v14, mai formája v23). 4-es mélységtől a gyökérkeresés nem $(-\infty, +\infty)$ ablakkal indul, hanem az előző iteráció értéke körüli $\pm\delta$ ablakkal, ahol $\delta = 25 + |s|/8$ centigyalog. Ha az eredmény az ablakon kívül esik, a kereső a megfelelő irányban duplázza az ablakot, és újrakeres.

Gyökérlépés-rendezés és fővváltozat (v17). A gyökérben a lépéseket az előző iteráció pontszáma szerint rendezi. A fővváltozatot (PV) egy PV-tábla gyűjti, amelyet a BuildPvString a transzpozíciós táblából legális lépésekkel tovább bővít, ez jelenik meg az info kimenetben.

Matt-távolság vágás és döntetlenek (v1). Ha már egy rövidebb matt ismert, az ennél hosszabb mattot kereső ágak azonnal levághatók. A keresés minden csomópontban ellenőrzi az ötvenlépéses szabályt, az elégtelen anyagot és az ismétlést; a fán belül már a kétszeri ismétlés is döntetlen (az 5. fejezet).

6.2. Transzpozíciós tábla (v3, klaszterek v13, zármentes v23)

A transzpozíciós tábla (TT) 128 MB méretű, kettő hatványa számú, 64 bájtra igazított klaszterből áll. Egy klaszter négy 16 bájtos bejegyzést tartalmaz, így egy lekérdezés egyetlen gyorsítótár-sort érint. A bejegyzés egy 64 bites kulcsot és egy 64 bites, tömörített adatot tárol: érték (16 bit), lépés (16 bit), bábutípus, mélység, korlát típusa (pontos, alsó vagy felső korlát) és generáció (8–8 bit).

A többszálú kereséshez a tábla zármentes: a kulcs helyett a kulcs XOR adat értéket tárolja. Ha két szál egyszerre írja ugyanazt a bejegyzést, és az olvasó egy félig frissített állapotot lát, a XOR-ellenőrzés nem egyezik, és a bejegyzés egyszerűen érvénytelennek számít. Ez lényegesen gyorsabb a korábbi (v16–v20) mutex-, illetve spinlock-alapú megoldásnál.

A csere sorrendje: üres hely, azonos kulcsú bejegyzés, végül a legkisebb „$4 \cdot \text{kor} - \text{mélység}$” értékű bejegyzés. A matt-értékeket a tárolás a gyökértől mért távolságra korrigálja. TT-vágás csak nem-PV csomópontban történik, és nem, ha a félléptetés-számláló már 90 felett van (az ötvenlépéses szabály közelében a tárolt érték félrevezető lehet). A tábla a következő csomópont klaszterét előre betölti (prefetch), mielőtt a lépést végrehajtaná.

6.3. Lépésrendezés

Az alfa-béta hatékonysága szinte teljesen a lépésrendezésen múlik. A lépéseket a kereső pontozza (a 6.1. táblázat), majd nem rendezi az egész listát, hanem mindig csak a következő legjobbat választja ki (lazy pick-best, v17), mert egy vágás után a többi lépést úgysem kell megvizsgálni.

Lépés típusa	Pontszám	Verzió
TT-lépés	10 000 000	v3
Ütés vezérré/Amazonná vagy huszárrá változással	9 000 000 / 8 000 000	v12
Nem vesztes ütés (MVV-LVA, SEE ≥ 0)	$5\,000\,000 + 16 \cdot$ áldozat $-$ támadó	v1/v12
Csendes átváltozás vezérré/Amazonná vagy huszárrá	4 000 000 / 3 000 000	v12
1. és 2. gyilkos lépés	2 000 000 / 1 500 000	v5
Ellenlépés (countermove)	1 200 000	v6
Csendes lépés	történeti érték	v6
Vesztes ütés (SEE < 0)	$-1\,000\,000$	v12
Bástyává/futóvá alulátváltozás	$-2\,000\,000$	v12

6.1. táblázat. A lépések rendezési pontszámai.

MVV-LVA és SEE. Az ütéseket elsősorban az ütött bábu értéke, másodsorban az ütő bábu olcsósága szerint rendezi (Most Valuable Victim – Least Valuable Attacker). Ha az ütő értékesebb az áldozatnál, a statikus cserekiértékelés (SEE) [8] dönti el, hogy a cserelánc a végén nyereséges-e: a mező összes támadóját a legolcsóbbtól kezdve felváltva üti le, a röntgentámadásokat (pl. bástya a bástya mögött) is figyelembe véve.

Gyilkos lépések (v5) és ellenlépés (v6). Egy csendes lépés, amely egy testvércsomópontban vágást okozott, valószínűleg itt is jó: ezt féllépésenként két gyilkos lépés tárolja. Az ellenlépés-heurisztika az ellenfél előző lépésére (bábutípus és célmező) adott legjobb választ jegyzi meg.

Történeti heurisztika (v6, gravitáció v18). Minden csendes lépés (szín, honnan, hová) kap egy pontszámot, amely vágás esetén nő, a vágás előtt kipróbált, sikertelen

csendes lépéseknél pedig csökken. A gravitációs frissítés a pontszámot korlátok között tartja, így a régi információ fokozatosan elévül.

6.4. Vágások, redukciók és kiterjesztések

A 6.2. táblázat a végleges kereső összes vágási és redukciós technikáját foglalja össze; a feltételek a nem-PV csomópontokra vonatkoznak, ha nincs sakk.

Technika	Működés (végleges paraméterek)	Verzió
Fordított futility	ha $\text{eval} - (100d - 75 \cdot \text{javuló}) \geq \beta$ és $d \leq 6$, a csomópont azonnal elvágható	v9
Razoring	ha $d \leq 3$ és $\text{eval} + 200 + 150d < \alpha$, csak nyugalmi keresés dönt	v9
Null-lépés vágás	a lépésjogot átadva, $R = (823 + 67d)/256 + \min((\text{eval} - \beta)/200, 3)$ redukcióval keres; ha az ellenfél így sem éri el β-t, vágás; gyalogvégjátékban tiltott (zugzwang)	v8
Futility vágás	$d \leq 6$ esetén a csendes lépések kihagyása, ha $\text{eval} + 120d \leq \alpha$	v10
Késői lépések vágása	$(\text{javuló?}4 : 2) + d^2$ csendes lépés után a többi kihagyása, $d \leq 8$	v11
SEE-vágás	$d \leq 7$ esetén a $-50d$ (csendes) vagy $-90d$ (ütés) alatti SEE-jű lépések kihagyása	v11
LMR	a késői, csendes lépések csökkentett mélységű keresése (lásd lent)	v7
IIR	TT-lépés hiányában a mélység eggyel csökken	v23
Szinguláris kiterjesztés	ha a TT-lépés minden más lépésnél jelentősen jobb, +1 mélység; több vágó alternatíva esetén „multi-cut” vágás vagy -1 kiterjesztés	v23

6.2. táblázat. Vágások, redukciók és kiterjesztések (d: hátralévő mélység, eval: statikus értékelés).

A javulás jelző (v9). Ha a lépő fél statikus értékelése jobb, mint két félléptetéssel korábban, az állás javuló. Ilyenkor a kereső óvatosabban vág (nagyobb futility-ráhagyás, több lépés a késői lépések vágása előtt, kisebb LMR-redukció), hiszen az állás valószínűleg még tartogat valamit.

Késői lépések redukciója (v7, külön PV-tábla v11). A rendezés végén álló csendes lépések nagy valószínűséggel rosszak, ezért a kereső csökkentett mélységgel vizsgálja őket, és csak akkor keresi újra teljes mélységgel, ha meglepően jónak bizonyulnak. Az alapredukció egy előre kiszámított táblából jön: nem-PV csomópontban $0{,}5 + \ln d \cdot \ln m/2{,}5$, PV-ben $-0{,}25 + \ln d \cdot \ln m/3$, ahol m a lépés sorszáma. Ezt módosítja, hogy javuló-e az állás, gyilkos lépésről, szabad gyalog előretolásáról, sakkadásról, átváltozásról van-e szó, illetve a lépés történeti pontszáma (± 2).

Szinguláris kiterjesztés (v23). Ha 8-as mélység felett a TT-lépés értéke megbízható, a kereső a TT-lépés kihagyásával, csökkentett mélységgel és TT-érték $-(3 + d/2)$ korláttal ellenőrzi a többi lépést. Ha egyik sem éri el ezt, a TT-lépés az egyetlen jó lépés, ezért egy mélységgel tovább keresi; ha viszont már a korlát is vágást okoz, több jó lépés van, és a csomópont azonnal levágható.

Sakk-kiterjesztés. A korai verziók minden sakkadást egy mélységgel meghosszabbítottak. A v23 ezt kivette, mert a szinguláris kiterjesztés és a nyugalmi keresés a fontos sakkokat már lefedi, a kiterjesztés pedig a mérésekben inkább rontott.

6.5. Nyugalmi keresés (v1)

A mélységkorlát elérésekor a kereső nem azonnal értékel, hanem csak az ütéseket (és az erős átváltozásokat) vizsgálja tovább, amíg az állás nyugodt nem lesz. A lépő fél állva maradhat:

ha a statikus értékelés már eléri β-t, nem kell tovább keresni. A végleges változat elemei: kitérő lépések generálása sakkban (v2–v3), delta-vágás (v3; kihagyja azt az ütést, amely még a legnagyobb lehetséges nyereséggel sem érné el α-t), SEE-vágás (v11), TT-lekérdezés és -tárolás (v23), valamint a bástyává és futóvá alakuló átváltozások elhagyása.

6.6. Állásértékelés

NNUE HalfKP (v1). A kezdeti értékelés a Stockfish egy HalfKP formátumú hálója (`nn-62ef826d1a6d.nnue`) a `nnue-probe` könyvtáron keresztül. A kereső minden lépésnél csak a megváltozott bábukat (dirty pieces: mozgó bábu, ütött bábu, sáncnál a bástya, átváltozásnál a gyalog és az új bábu) adja át, a könyvtár ezekből inkrementálisan frissíti az akkumulátort.

NNUE HalfKAv2_hm (v23). A legfrissebb verzió egy újabb generációs, HalfKAv2_hm formátumú Stockfish-hálót (nn-dabb1ed23026.nnue) használ, amelynek kiértékelését saját kóddal valósítottam meg (nnue-halfkav2). A háló 22 528 bemeneti jellemzőt (király helye $\times$ bábu $\times$ mező, a király oldalára tükrözve), 2×1024 neuronos első réteget, 8 anyagi vödröt és vödrönként külön 15–32–1 neuronos réteghalmazt tartalmaz. A vödröt a táblán lévő bábuk száma választja ki, így a háló külön réteget használ a megnyitásra, a középjátékra és a végjátékra. Az akkumulátorok egy veremben, félléptetésenként inkrementálisan frissülnek, ha valamelyik király mozog, az adott nézőpont akkumulátorát a kód teljesen újraszámolja. A belső szorzások AVX2 utasításokkal futnak.

Kézzel írt értékelés. A gyengébb nehézségi szintek egy saját, kézzel hangolt értékelőt használnak: anyag és bábu-mező táblák középjátékra és végjátékra külön, játékfázis szerinti átmenettel, mobilitás, gyalogszerkezet (szabad, elszigetelt gyalogok), királybiztonság és gyalogpajzs, bábuegyensúly, ismert döntetlen végjátékok felismerése és a mop-up tag, amely nyert végjátékban a vesztes királyt a szélre szorítja. Ez gyengébb, mint az NNUE, de pont ez a cél: emberibb, hibázó játékot ad.

6.7. Időkezelés

Fix idős módban a kereső a megadott ideig gondolkodik. Órás módban két határt számol: a puha határ hátralévő idő/40+0,8·bónusz, a kemény határ ennek háromszorosa (mindkettő a hátralévő időhöz korlátozva). Új iterációt csak a puha határ előtt kezd; ha a legjobb lépés háromszor egymás után nem változott, a puha határ 60%-ánál is megáll. Válság esetén, ha az érték 50 centigyalognál többet esett, de az állás még nem dőlt el ($\pm$800 centigyalogon belül), a puha határon túl is folytatja, a kemény határ 80%-áig.

6.8. Többszálú keresés: Lazy SMP (v16)

A kereső a beállított számú szálon fut. A segédszálak ugyanazt az állást keresik, saját előzménytáblával és lépésrendezéssel, de közös transzpozíciós táblán keresztül. Mivel a szálak kissé eltérő sorrendben haladnak, egymás számára hasznos TT-bejegyzéseket hagynak hátra, így a fő szál gyorsabban ér el egy adott mélységet. A szálak egy közös megállító jelzőn keresztül állnak le, az eredményt mindig a fő szál adja.

6.9. Nehézségi szintek

A négy nehézségi szint (a 6.3. táblázat) nem egyszerűen a keresési mélységet korlátozza. Az alsó három szint kézzel írt értékelést használ, és a `GetBestAmongTopMoves` egy több jelöltlépést kiértékelő keresés után szándékos hibát is véthet: adott valószínűséggel a legjobb N lépés közül választ egy olyat, amely legfeljebb a küszöbnyit rosszabb a legjobbnál. A Haladó és Nehéz szint kiszűri a kínos hibákat (amikor a választott lépés után egy bábu védtelenül üthetővé válik), és egy ingyen kínálkozó bábut mindig leüt. Két szándékos hiba között legalább egy-két normál lépést tesz.

Szint	Max. mélység	Értékelés	Hiba esélye	Top N	Küszöb [cp]	Kínos hiba szűrése
Kezdő	6	kézzel írt	40%	bármelyik	900	nem
Haladó	6	kézzel írt	30%	10	500	igen
Nehéz	9	kézzel írt	20%	8	300	igen
Mester	–	NNUE	0%	–	–	–

6.3. táblázat. A nehézségi szintek paraméterei (`SearcherSettings`).

7. fejezet

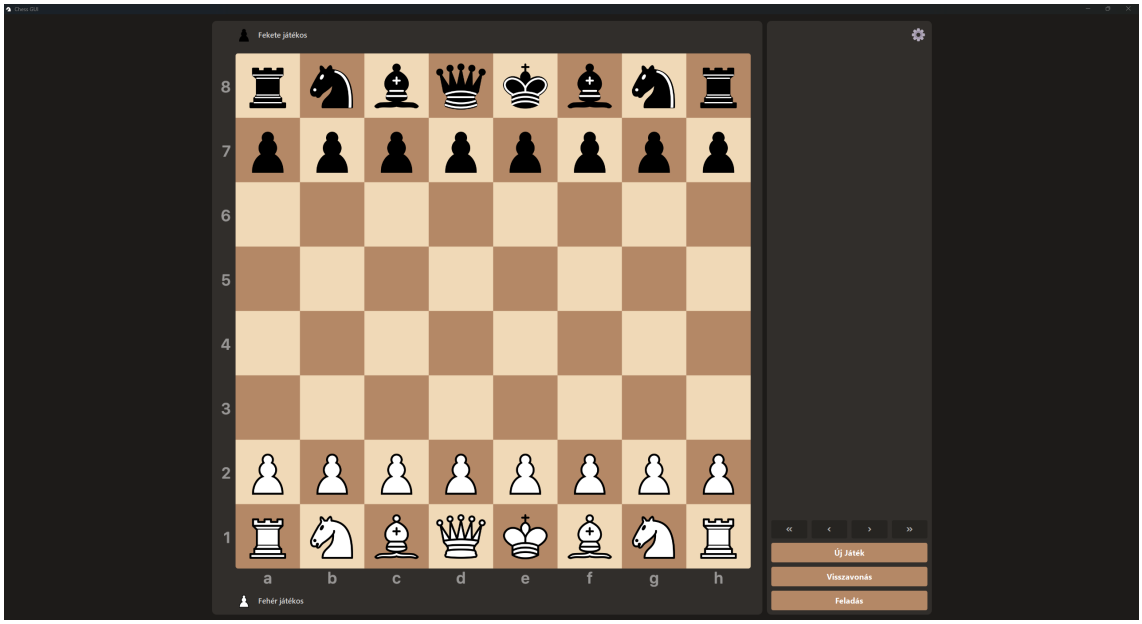
A grafikus felhasználói felület

7.1. Felépítés

A felület Qt 6-ban készült, a táblát a Qt Graphics View keretrendszere [21] rajzolja. A szerkezet a Model–View–ViewModel mintát követi:

- a *modell* a `BoardManager` és a motor (a 4. fejezet),
- a *nézetmodell* a `ChessViewModel`: lefordítja a felhasználói műveleteket (lépés, visszavonás, feladás, új játszma, beállítások) motorhívásokká, kezeli a robot keresőszálát és a sakkórák időzítőjét, és Qt-jelzésekkel (`boardChanged`, `moveMade`, `gameEnded`, …) értesíti a nézeteket,
- a *nézetek*: a táblát rajzoló `ChessScene`/`ChessView`, a lépéslistát és a vezérlőgombokat tartalmazó `InfoView`, a játékosok adatait, óráit és a leütött bábukat mutató `BoardAndPlayerPanel`, valamint a `SettingsDialog`.

A nézetek soha nem érik el közvetlenül a motort, így a tábla rajzolása és a keresés egymástól teljesen független.



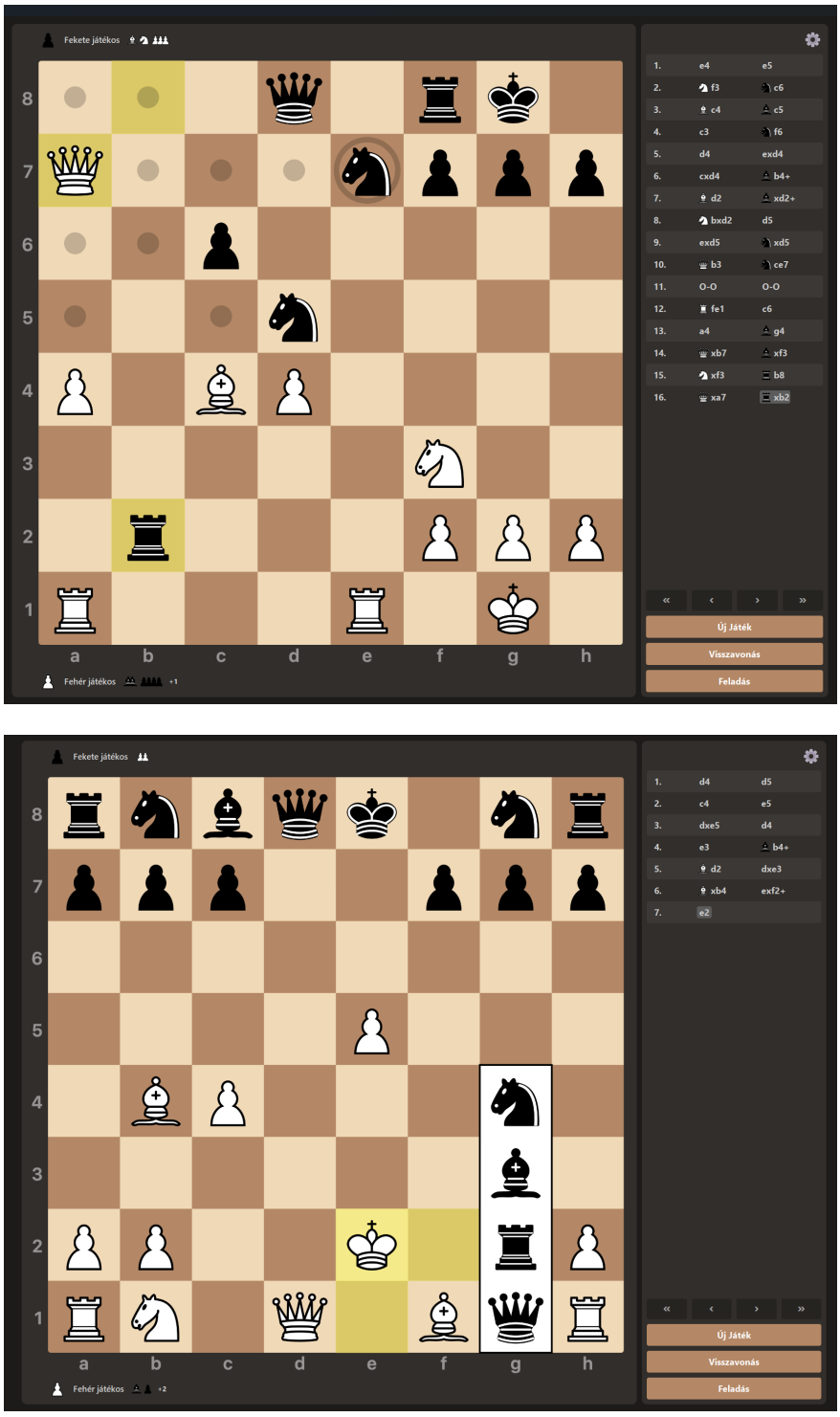
7.1. ábra. A program főablaka.

7.2. A főablak

A 7.1. ábrán látható főablak bal oldalán a tábla, felette és alatta a két játékos panelje áll. A panel a játékos nevét (robot esetén a kereső nevét és nehézségi szintjét), ikonját, a sakkórát, a leütött ellenséges bábukat és az anyagi előnyt (pl. „+3”) mutatja. A jobb oldali információs panelen a játszma lépései SAN-jelöléssel, a mozgó bábu ikonjával jelennek meg. A lépéslista alatt navigációs gombokkal («, ‹, ›, ») vagy egy lépésre kattintva a játszma bármely korábbi állása visszanézhető; ilyenkor a tábla és az órák a régi állapotot mutatják. Az „Új Játék”, „Visszavonás” és „Feladás” gombok, valamint a fogaskerék ikonú beállítások gomb zárják a panelt. A játszma végén a panel az eredményt (pl. „1 – 0, Világos nyert”) is kiírja.

7.3. Interakció a táblán

A bábuk egérrel húzhatók (drag and drop), vagy kattintással kijelölhetők, majd a célmezőre kattintva léptethetők. Kijelöléskor a felület pontokkal jelöli a szabályos célmezőket (ez kikapcsolható), kiemeli a legutóbbi lépés két mezőjét, sakk esetén pirossal a megtámadott királyt, és az egér alatti mezőt is. A szabályos lépések listáját a motor számolja, a felület csak megjeleníti, így a felület soha nem enged szabálytalan lépést. Átváltozáskor a célmező oszlopában négy választható bábu jelenik meg (7.2. ábra). Robot ellen játszva a robot a felhasználó lépése után automatikusan, külön szálon gondolkodik, ha a felhasználó közben visszavon vagy új játszmát indít, a keresés megszakad.



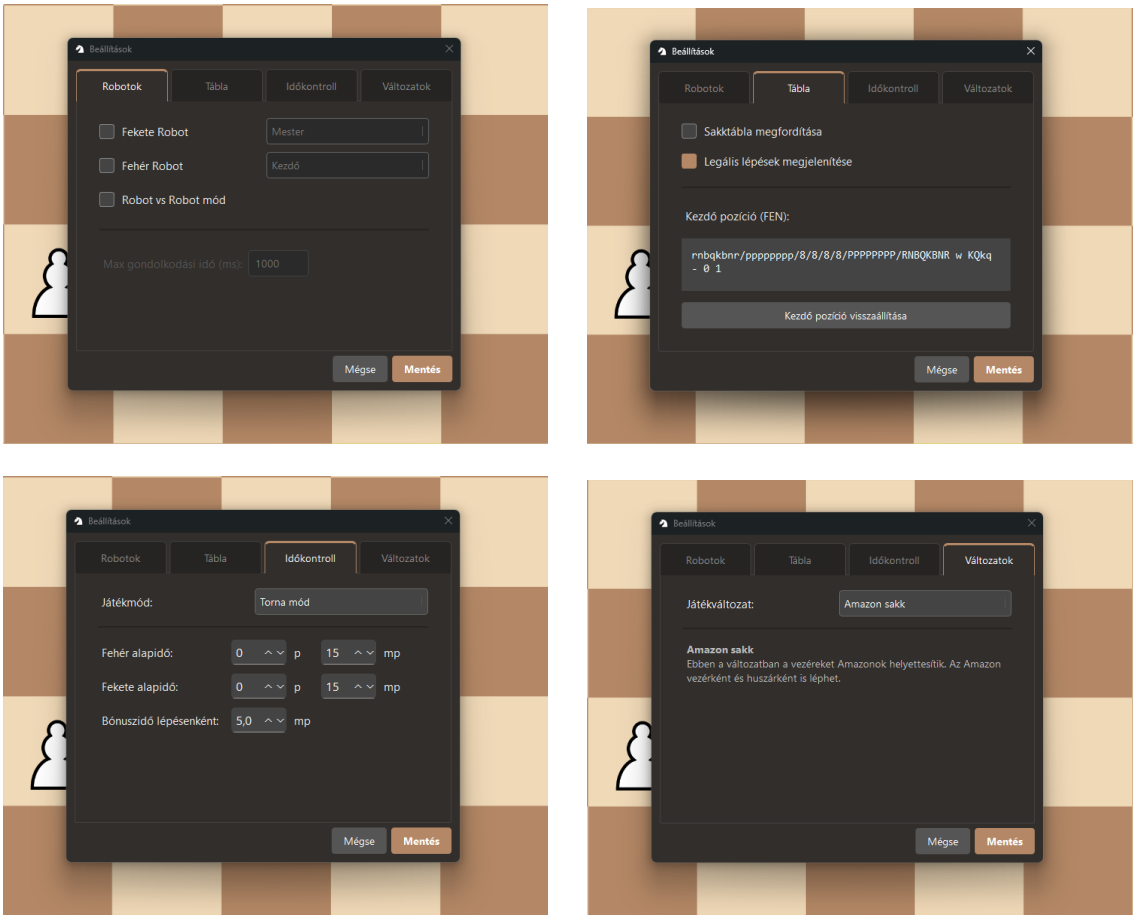
7.2. ábra. Lépés közben megjelenített segítségek (fent) és az átváltozás bábuválasztója (lent).

7.4. Beállítások

A beállítások ablak négy fülön csoportosítja a lehetőségeket (7.3. ábra):

- *Robotok:* külön-külön beállítható, hogy világos és/vagy sötét robot legyen, és milyen nehézségi szinten (Kezdő, Haladó, Nehéz, Mester) játsszon; itt kapcsolható be a robot-robot mód és állítható a robot maximális gondolkodási ideje.
- *Tábla:* tábla megfordítása, a szabályos lépések jelölése és tetszőleges kezdőállás megadása FEN-ben.
- *Időkontroll:* óra nélküli vagy „torna” mód, a két fél alapideje külön-külön, és a lépésenkénti bónuszidő.
- *Változatok:* a játékváltozat (Normál vagy Amazon sakk) kiválasztása (a 8. fejezet).

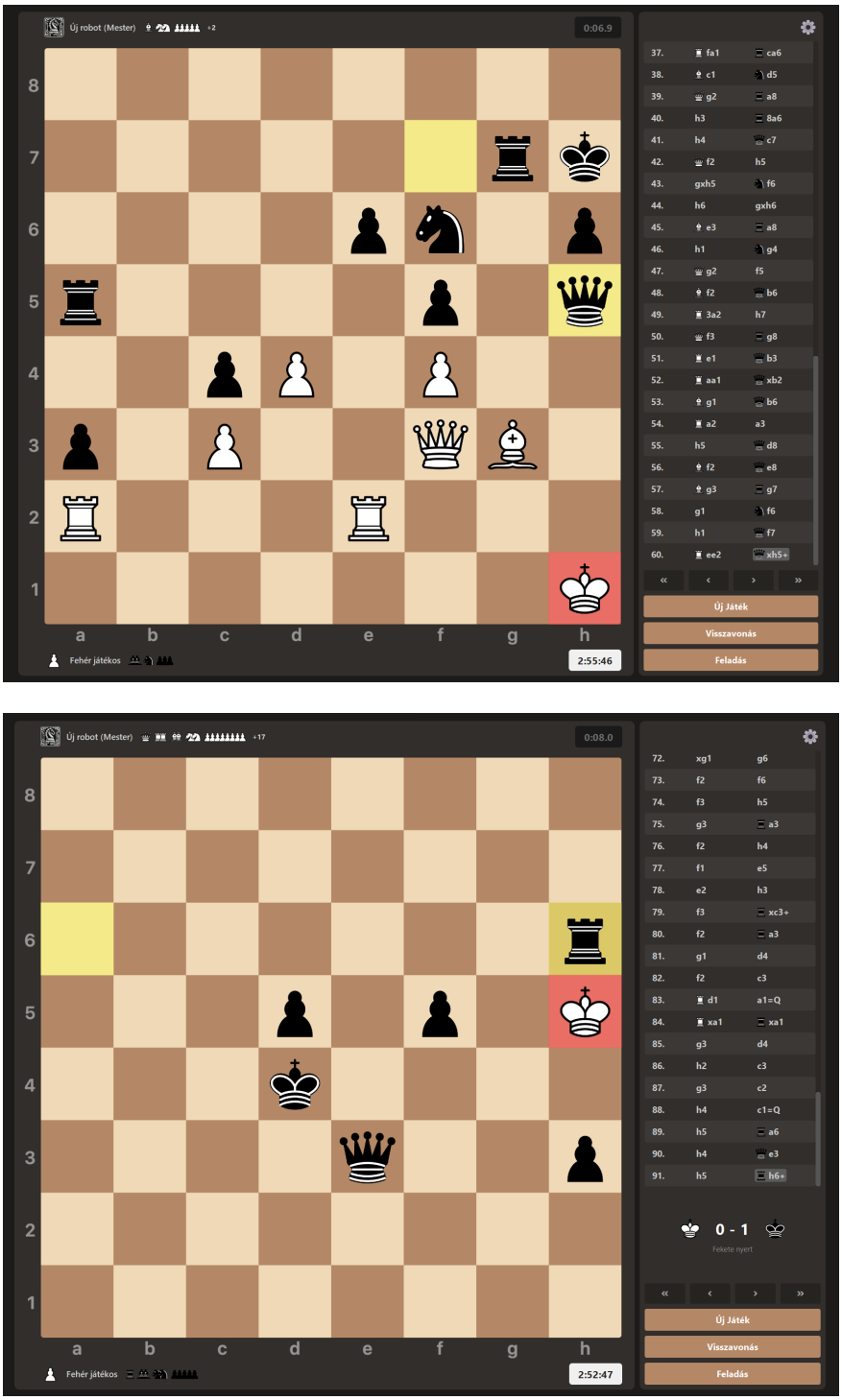
A dialógus csak a „Mentés” gombra írja vissza az értékeket; a nézetmodell ezután összeveti a régi és új beállításokat, és csak akkor indít új játszmát, ha erre szükség van (pl. megváltozott a kezdőállás vagy a robot típusa).



7.3. ábra. A beállítások ablak négy füle.

7.5. Játék sakkórával és robot-robot mód

Torna módban mindkét fél órája a beállított alapidőről indul, a lépő fél órája másodpercenként frissül, lépés után pedig a bónuszidővel nő. Az órák állását a játékvezérlő minden félléptetés után elmenti, így visszavonáskor az órák is visszaállnak. Robot-robot módban a felület a megnyitáskönyvből véletlen kezdőállást választ, és a két kereső egymás ellen játszik, színcserés párokban; ez a mód a fejlesztés során a motor viselkedésének szemmel követésére szolgált (7.4. ábra).



7.4. ábra. Sakkórás játszma robot ellen (fent) és egy befejezett játszma eredménykijelzése (lent).

8. fejezet

Tündérsakk-kiterjesztés: az Amazon sakk

8.1. A változat szabályai

Az „Amazon sakk” változatban mindkét fél vezérének helyén (d1, d8) egy Amazon áll, amely vezérként és huszárként is léphet (a 2.1. ábra). A változatban vezér egyáltalán nincs: a gyalog átváltozáskor huszárrá, futóvá, bástyává vagy Amazonná változhat. Minden más szabály (sáncolás, en passant, döntetlenek) változatlan. A kezdőállás FEN-jelöléssel `rnbakbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBAKBNR w KQkq - 0 1`, ahol az Amazon jele `A`, illetve `a`.

8.2. Megvalósítás a motorban

A kiterjesztés célja az volt, hogy a meglévő kódhoz minél kevesebb, jól körülhatárolt módosítással illeszkedjen.

Bábutípus és állapot. Az `AMAZON` új bábutípusként a `KING` után került a felsorolásba. Így a korábbi típusok indexei (és minden rájuk épülő tábla, pl. bábu-mező táblák) változatlanok maradtak, a bitboard-tömb és a Zobrist-kulcsok tömbje pedig automatikusan eggyel nőtt. Az összes „minden bábutípuson” végigmenő ciklus a `PIECE_TYPE_COUNT` határig fut, így a FEN-olvasás és -írás, a hash-számítás és a táblakép is kezeli az új bábut.

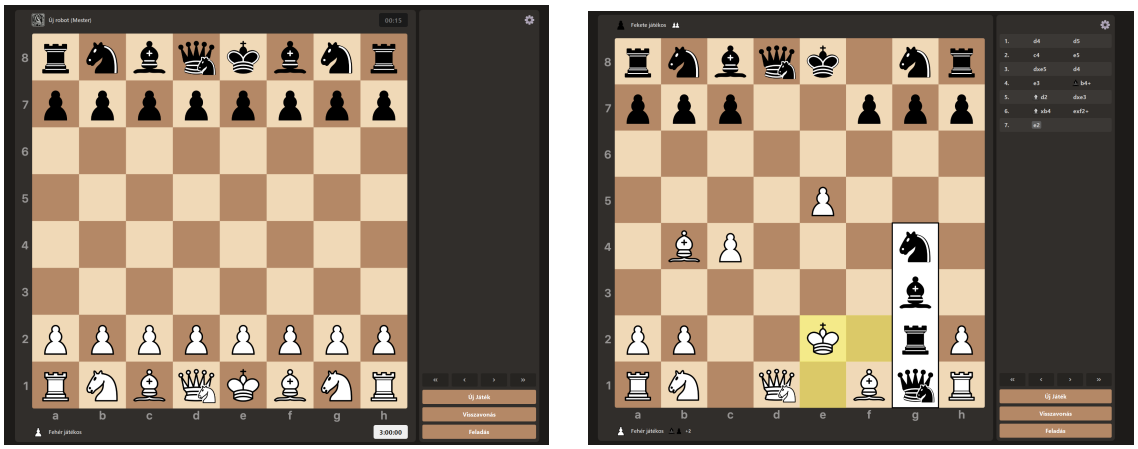
Lépésgenerálás és támadásszámítás. Az Amazon célmezői a bástya-, futó- és huszártámadások uniója. Mivel a sakkfigyelés fordított irányú (az 5. fejezet), elég volt az Amazon bitboardját a huszár- és mindkét csúszótámadás ellenőrzéséhez hozzáadni. Ugyanígy bővült a cserekiértékeléshez (SEE) használt `getAttacksTo`, a röntgentámadásokat adó `getBishopsQueens`/`getRooksQueens`, valamint a „legkisebb értékű támadó” sorrend (gyalog, huszár, futó, bástya, vezér, Amazon, király).

Átváltozás. A lépéskódolásban az átváltozás két bitje pontosan négy bábut ír le, és mind a 16 jelzőkombináció foglalt. Mivel az Amazon változatban vezér nincs, a tábla ismeri az aktuális változatot, és ilyenkor a „vezérré változás” kódú lépés Amazont tesz le. Így sem a lépésformátum, sem a transzpozíciós tábla nem változott. A SAN-jelölés ilyenkor `=A`, a felület bábuválasztója pedig a vezér helyett az Amazont kínálja (8.1. ábra).

Értékelés. Mindkét NNUE-hálót klasszikus sakkra tanították, így Amazont nem ismernek. A megoldás: az értékelés az Amazont vezérként adja a hálónak (a HalfKP-kódolásban

és a HalfKAv2 bábukódjában is), majd a kereső a háló eredményéhez Amazononként +300 centigyalogot ad a lépésre következő fél szemszögéből (saját Amazon +300, ellenséges −300). A kézzel írt értékelés ugyanezt teszi egy segédfüggvényen keresztül, amely minden bitboard-lekérdezésnél az Amazonokat a vezérekhez sorolja. A lépésrendezés és a SEE az Amazont $900+300=1200$ értékkel számolja, az Amazonná változás ugyanolyan prioritást kap, mint a vezérré változás, és a nyugalmi keresés sem dobja el.

Felület. A beállítások „Változatok” fülén választható a játékváltozat; a változtatás új játszmát indít a változat kezdőállásával. A mentés ellenőrzi, hogy normál változatban ne legyen Amazon, Amazon változatban pedig vezér a megadott FEN-ben. Az Amazon saját képet kapott (vezér, előtte kisebb huszárfejjel), és a leütött bábuk panelje is a változat kiinduló bábukészletéhez viszonyít.



8.1. ábra. Az Amazon sakk a grafikus felületen: kezdőállás (balra) és átváltozás (jobbra).

8.3. Ellenőrzés

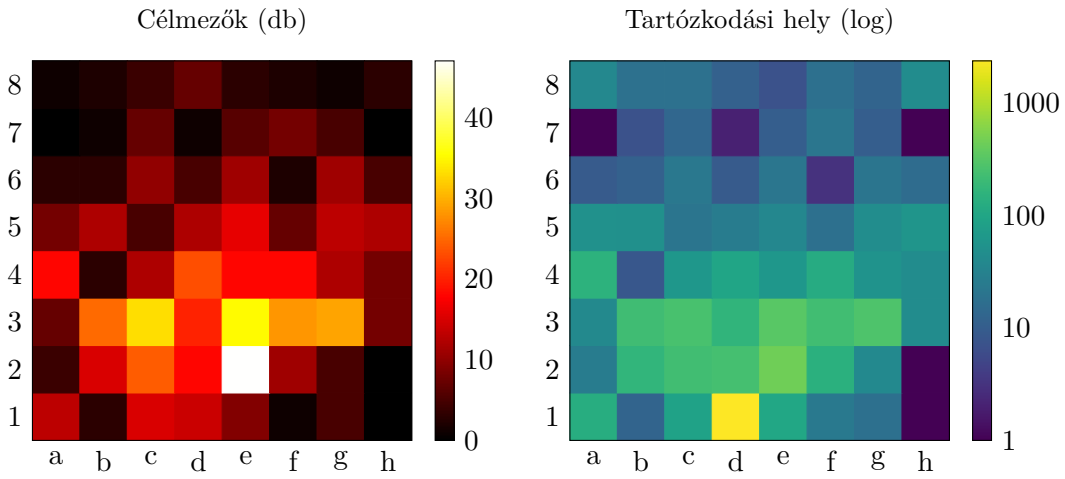
A klasszikus perft-értékek a módosítás után is pontosan egyeznek (a 9.1. táblázat), tehát a normál sakk működése nem változott. Az Amazon változatra nincs nyilvános referencia, ezért a helyességet célzott tesztekkel ellenőriztem: a kezdőállásban 22 szabályos lépés van (20 + az Amazon két huszárugrása c3-ra és e3-ra), egy üres tábla d4 mezőjén álló Amazon 35 mezőt ér el, az egyedül álló Amazon mattot ad, a perft minden csomópontjában az inkrementális és a teljes újraszámolt Zobrist-kulcs megegyezik, és az átváltozás utáni visszavonás is pontosan visszaállítja az állást. Az Amazon kezdőállás perft-értékei: 22, 484, 12 483, 318 185 és 9 319 911 (1–5 mélység).

8.4. Hogyan játszik a kereső Amazonnal?

A kereső viselkedését 64 önmaga elleni játszmán vizsgáltam (lépésenként 200 ms, „Mester” szint, az első két lépés véletlenszerű, hogy a játszmák különbözzenek, ezek nem számítanak a statisztikába). Az eredmény 18 világos és 10 sötét győzelem, 33 döntetlen, 3 játszma pedig elérte a 300 félléptetéses korlátot. Néhány megfigyelés:

- *Kedvenc nyitólépés:* a kezdőállásból 30 független, 1 másodperces keresésből 28-szor `e2e4`, kétszer `d2d4` volt a választás. Az e-gyalog előlépése az e2 mezőt is felszabadítja, amely az Amazon leggyakoribb célmezője.

- *Mozgásmód:* az összes lépés 6,6%-át (667 lépés) tette Amazon; ezek 28,5%-a huszárugrás, 71,5%-a vezérszerű lépés volt, és 118 alkalommal adott sakkot.
- *Csere:* az Amazon leggyakoribb ütött bábuja a gyalog (63) után az ellenséges Amazon (54). A 128 Amazonból 115 a játszma során lekerült a tábláról, átlagosan az 51. félléptetésben: a kereső egyértelműen törekszik az Amazonok cseréjére, ha az nem hátrányos.



8.2. ábra. Az Amazon hőtérképe 64 önjátszmából (a sötét Amazon mezőit tükrözve, mindkettő világos szemszögéből). Balra: hová lépett az Amazon, jobbra: mely mezőkön állt félléptetésenként (logaritmikus skála).

A 8.2. ábra szerint az Amazon leggyakoribb célmezői az e2 (47), e3 (35), c3 (33), g3 (29) és f3 (28): a kereső a megnyitásban nem rohamoz vele, hanem a második-harmadik sorban, a király közelében fejleszti, ahonnan huszárugrással és csúszó lépéssel egyaránt sok mezőt fenyeget. A tartózkodási térképen a d1 kiinduló mező dominál, hiszen az Amazon a játszmák elején gyakran sokáig a helyén marad, majd a tábla saját térfelére eső részén mozog; az ellenfél térfelén csak ritkán és rövid ideig tartózkodik.

9. fejezet

Mérések és értékelés

A méréseket egy Intel Core i7-10700 processzoros (8 mag, 16 szál, 2,9 GHz), 32 GB memóriájú asztali gépen végeztem (Windows 11, MinGW-w64 GCC 13.1, `-O3 -march=native -flto`, a GUI projekt kiadási beállításaival megegyezően). A mérőprogram a motor forrásaiból, a felület nélkül fordult.

9.1. A lépésgenerálás helyessége és sebessége

A lépésgenerátort öt, a szakirodalomban szabványos tesztállással ellenőriztem [6]. A kezdőállás mellett a „Kiwipete” (sáncolás, en passant és átváltozás egyszerre), a „3. állás” (végjáték, en passant-kötések), a „4. állás” (sakkból kilépő átváltozások) és az „5. állás” (sáncolás és átváltozás csapdák) szerepel. A 9.1. táblázat szerint a motor minden mélységen pontosan a referenciaértéket adja, összesen több mint 400 millió végállásra.

Állás	Mélység	Csomópontok	Idő [s]	Csomópont/s	Referencia
Kezdőállás	5	4 865 609	0,157	31,0 M	egyezik
Kezdőállás	6	119 060 324	3,948	30,2 M	egyezik
Kiwipete	4	4 085 603	0,125	32,6 M	egyezik
Kiwipete	5	193 690 690	5,798	33,4 M	egyezik
3. állás	6	11 030 083	0,434	25,4 M	egyezik
4. állás	5	15 833 292	0,508	31,2 M	egyezik
5. állás	5	89 941 194	2,839	31,7 M	egyezik
Amazon kezdőállás	5	9 319 911	0,306	30,4 M	(saját teszt)

9.1. táblázat. Perft-eredmények egy szálon (minden kisebb mélység is egyezik a referenciával).

A lépésgenerátor egy szálon másodpercenként 25–33 millió állást jár be. Ez a szám tartalmazza minden lépés teljes végrehajtását, a legalitás ellenőrzését és a visszavonást, tehát a keresésben ténylegesen használt utat méri. Az Amazon változat sebessége a klasszikuséval azonos nagyságrendű, a többletbábu nem lassította érdemben a generátort.

9.2. A döntéshozatal hatékonysága

A 9.2. táblázat a „Mester” szintű kereső mélységét és taktikai pontosságát mutatja. A taktikai állások a klasszikus *Win At Chess* (WAC) feladatgyűjtemény [16] első darabjai, ahol a helyes lépés ismert.

A hat feladatból ötöt a kereső a könyvbeli lépéssel oldott meg, a mattokat jellemzően ezredmásodpercek alatt. A WAC.002 esetén a motor más lépést választott, de saját ér-

Feladat	Szálak	Idő	Elért mélység	Eredmény
Kezdőállás	1	1 s	15	`e2e4`
Kezdőállás	7	1 s	15	`d2d4`
Kezdőállás	7	5 s	20	`d2d4`
Kiwipete	7	1 s	20	`e2a6` (Fxa6)
Matt 1 lépésben	7	1 s	1	`d1d8#` helyes, 1 ms alatt
WAC.001	7	2 s	14	`g3g6` helyes, matt 2-ben, 69 ms
WAC.002	7	2 s	25	`b3b7` (a könyvbeli `b3b2` helyett)
WAC.003	7	2 s	20	`e3g3` helyes
WAC.004	7	2 s	9	`h6h7` helyes, matt 2-ben, 14 ms
WAC.005	7	2 s	1	`c6c4` helyes, matt 2-ben, <1 ms

9.2. táblázat. Keresési mélység és taktikai feladatok („Mester” szint, HalfKAv2 értékelés).

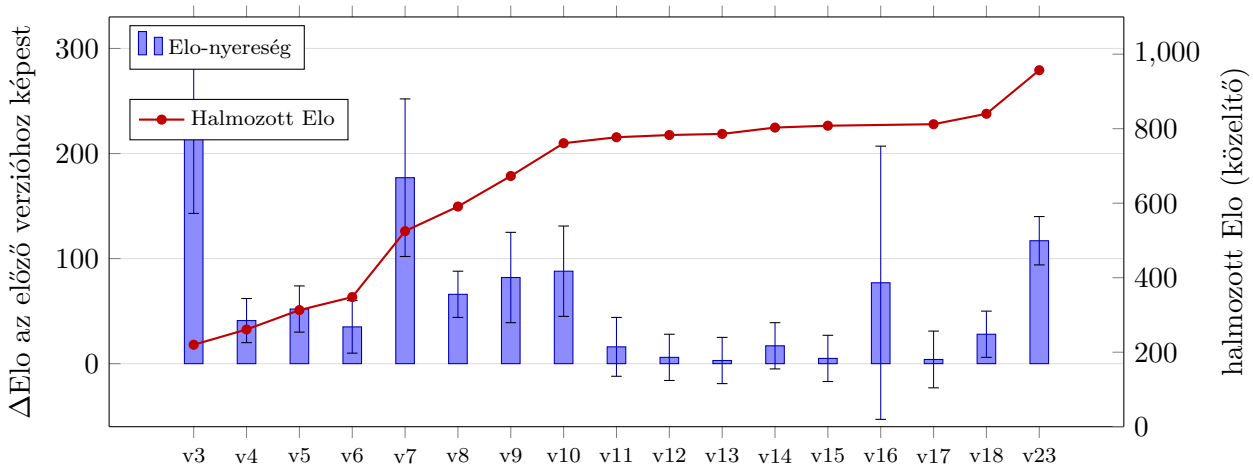
tékelése szerint ez is nyerő (sötét szemszögéből kb. +10,8 gyalogegység). A kezdőállásból egy szálon egy másodperc alatt 15-ös mélységet ér el, ami a vágások és a lépésrendezés hatékonyságát mutatja: a 15 mélységű teljes fa bejárása ennél nagyságrendekkel több időt igényelne.

9.3. A kereső fejlődése verzióról verzióra

A kereső minden jelentős módosítását robot-robot mérés követte: az új verzió (Improved searcher) a korábbi verzió (Old searcher) ellen játszott, a megnyitáskönyv véletlen kezdőállásaiból, színcserés párokban (a 4.4. szakasz). A 9.3. táblázat minden mérést felsorol a vizsgált változtatással együtt. A pontszám a győzelmek és a döntetlenek felének aránya, az Elo-különbség ebből a szokásos logisztikus modellel számolható:

$$\Delta\mathrm{Elo} = -400 \log_{10}\left(\frac{1}{s} - 1\right), \tag{9.1}$$

ahol s a pontszázalék.



9.1. ábra. Az egyes verziók erősödése az előző verzióhoz képest (oszlopok) és az ebből halmozott, közelítő Elo-nyereség (vonal). A v16 mérés csak 32 játszmából áll, ezért a halmozott értékbe nem számít bele.

Mérés	Dátum	Vizsgált változtatás	Gy/D/V	Pont	ΔElo
v3 – v2	04.24.	transzpozíciós tábla, delta-vágás a nyugalmi keresésben	87/18/18	78,0%	+220 [+153; +306]
v4 – v3	04.25.	fail-soft vágások, gyökér a negamaxban	379/360/261	55,9%	+41 [+20; +63]
v5 – v4	04.25.	gyilkos lépések	391/366/243	57,4%	+52 [+30; +74]
v6 – v5	04.25.	történeti heurisztika	260/251/189	55,1%	+35 [+10; +61]
v7 – v6	04.25.	PVS és késői lépések redukciója (LMR)	61/41/9	73,4%	+177 [+109; +259]
v8 – v7	04.25.	null-lépés vágás	322/545/133	59,5%	+66 [+45; +89]
v8 – v8	04.26.	ellenőrző mérés: azonos kód önmaga ellen	28/51/27	50,5%	+3 [−64; +70]
v9 – v8	04.26.	fordított futility (statikus null-lépés), „javuló” jelző	102/124/40	61,7%	+82 [+41; +127]
v10 – v9	04.26.	futility vágás	102/138/34	62,4%	+88 [+47; +132]
v11 – v10	04.26.	SEE-vágás a nyugalmi keresésben, SEE újraírása	131/405/101	52,4%	+16 [−11; +44]
v12 – v11	04.27.	vesztes ütések és alulátváltozások hátrasorolása	188/654/170	50,9%	+6 [−15; +28]
v13 – v12	04.27.	4 elemes TT-klaszterek, előtöltés, időkezelés	169/670/161	50,4%	+3 [−19; +24]
v14 – v13	04.28.	aspirációs ablak	245/575/194	52,5%	+17 [−4; +39]
v15 – v14	04.28.	SEE-javítás, ismétlésfigyelés a táblában	224/567/209	50,7%	+5 [−16; +27]
v16 – v15	04.28.	Lazy SMP többszálú keresés (csak 32 játszma)	11/17/4	60,9%	+77 [−42; +218]
v17 – v15	05.02.	IID, LMR-módosítók, gyökérlépés-rendezés, lusta lépésválasztás	160/362/153	50,5%	+4 [−23; +30]
v18 – v17	05.30.	történeti „gravitáció” és büntetés, history-tag az LMR-ben	220/670/137	54,0%	+28 [+7; +50]
v23 – today	10.12.	zármentes TT, TT a nyugalmi keresésben, IIR, teljes fail-soft, új aspiráció	360/605/35	66,2%	+117 [+95; +140]

9.3. táblázat. Az összes verziómérés (2026). Gy/D/V: az új verzió győzelmei, döntetlenjei és vereségei.

A 9.1. ábra jól mutatja a fejlesztés két szakaszát. Az első napokban az alapvető technikák (TT, PVS és LMR, null-lépés, futility-vágások) egyenként is 35–220 Elo-pontot hoztak. A v11–v17 közötti módosítások már csak néhány Elo-pontos, a mérési hibán belüli javulást adtak; ezek nagy része tisztán sebesség- vagy robusztussági javítás volt (pl. TT-klaszterek, SEE-hibák javítása), amelyek a játékerőn rövid időkontrollal alig látszanak. A legnagyobb késői ugrást a v23 hozta: a zármentes transzpozíciós tábla, a nyugalmi keresésben is használt TT és az IIR együtt +117 Elo-pontot ért a közvetlen előd ellen. A közelítő halmozott nyereség a v2-től a v23-ig mintegy 950 Elo-pont; ez csak tájékoztató érték, mert a láncban a mérések nem mindig közvetlenül egymást követő verziók között történtek.

A legkorábbi, még a felülettel együtt fejlesztett keresőt (a v21 és v22 archív verziót, 2026. április 21.) a fenti sorozat nem tartalmazza: ezeket csak néhány tucat, illetve négy próbajátszmával mértem, és a v1-ben a keresőt tudatosan egy minimális magról, lépésenként újraépítettem, hogy minden technika hatása külön mérhető legyen.

9.4. Abszolút erősség: mérés a Stockfish ellen

A verziók közötti mérések csak relatív erősséget adnak. Az abszolút erősség becsléséhez a motort ismert erősségű ellenfél ellen kell játszatni. Erre a Stockfish 17.1 [23] beépített erősségkorlátozását használtam (`UCI_LimitStrength`, `UCI_Elo`), amely a dokumentáció szerint a CCRL 40/4 (blitz) ranglistához [2] van kalibrálva, 1320 és 3190 Elo között. A játszmákat a *cutechess-cli* [20] vezérelte, a motor a konzolos backend UCI-kiszolgálóján keresztül játszott. Mindkét motor egy szálon, 128 MB hash-táblával, 10 s + 0,1 s időkontrollal futott; a kezdőállások a saját megnyitáskönyv véletlen állásai voltak, mindegyikből

két játszmával, színcserével. A 300 játszma alatt egyetlen időtúllépés, szabálytalan lépés vagy összeomlás sem történt.

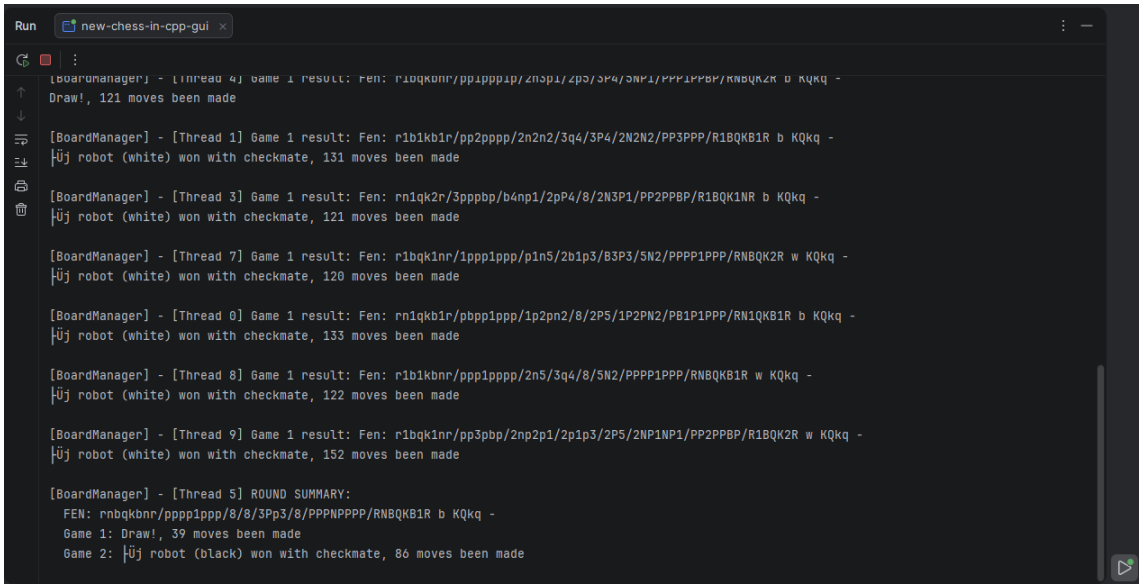
Ellenfél (Stockfish 17.1)	Gy / V / D	Pontszám	ΔElo	Becsült erősség
`UCI_Elo` = 2600	92 / 5 / 3	93,5%	$+463 \pm 157$	≈ 3063
`UCI_Elo` = 2900	62 / 17 / 21	72,5%	$+168 \pm 67$	≈ 3068
`UCI_Elo` = 3190	15 / 34 / 51	40,5%	-67 ± 48	≈ 3123

9.4. táblázat. A motor („Mester” szint, HalfKAv2) eredményei a korlátozott erősségű Stockfish ellen, szintenként 100 játszma (Gy/V/D: a saját motor győzelmei, vereségei és döntetlenjei).

A 9.4. táblázat szerint a 2600-as szint ellen a motor szinte minden játszmát megnyert; ilyen szélsőséges pontszámból az Elo-különbség csak pontatlanul becsülhető. A két informatív mérés (2900 és 3190) inverz varianciával súlyozott átlaga alapján a motor ereje **kb. 3100 ± 40 Elo a CCRL blitz skálán**, és a három mérés egymással összhangban van. Az eredmény értelmezésénél figyelembe kell venni, hogy a Stockfish korlátozását 120 s + 1 s időkontrollal kalibrálták, a mérés viszont gyorsabb időkontrollal készült, ahol a korlátozott Stockfish valamivel gyengébben játszhat a névleges értékénél; másrészt a saját motor egy szálon futott, így több szálon ennél erősebb. A CCRL-értékek nem vethetők össze közvetlenül a FIDE-értékszámokkal, de ezen a szinten a motor bármely emberi játékosnál lényegesen erősebb.

9.5. Mérések a konzolos backendben

A végleges kereső erejét a konzolos backend robot-robot szimulációjával mértem két időkontrollal: mindkét félnek 10 másodperc alapidő lépésenként 0,1 másodperc bónusszal, illetve 60 másodperc alapidő 0,6 másodperc bónusszal. A szimuláció a megnyitáskönyvből választott kezdőállásokból színcserés párokban játszik, és a végén összesíti a győzelmeket, a döntetleneket és a pontszázalékot. A 9.2. ábra egy futás közbeni példakimenetet mutat.



9.2. ábra. Példa a backend robot-robot szimulációjának futás közbeni kimenetére.

10. fejezet

Összefoglalás és továbbfejlesztési lehetőségek

10.1. Az elvégzett munka értékelése

A feladatkiírás minden pontját teljesítettem:

- *FIDE-szabályok és tündérsakk:* a motor a klasszikus szabályokat teljes egészében megvalósítja (sáncolás, en passant, átváltozás, ismétlés, ötvenlépéses szabály, elégtelen anyag), a tündérsakk alapelveit és az Amazon bábut a 2. fejezet mutatja be.
- *Architektúra és UCI:* a rétegzett felépítésnek köszönhetően ugyanaz a motor fut konzolos UCI-kiszolgálóként és grafikus felülettel; a keresők egy közös interfész mögött cserélhetők és egymás ellen futtathatók.
- *Klasszikus és új bábuk:* az Amazon sakk változat a lépésgenerálástól az értékelésig minden réteget érint, mégis néhány jól körülhatárolt módosítással illeszkedett, és a klasszikus játék működését bizonyíthatóan nem változtatta meg.
- *Grafikus felület:* a Qt-alapú felület négy nehézségi szintet, sakkórát, visszavonást, visszajátszást és változatválasztást kínál.
- *Mérések:* a lépésgenerálás helyességét perft-tesztek, a döntéshozatal hatékonyságát taktikai feladatok, több mint tizenkétezer verziók közötti mérőjátszma, valamint a korlátozott Stockfish elleni mérés igazolja, amely szerint a motor ereje kb. 3100 Elo a CCRL blitz skálán.

A legfontosabb tanulság a mérésvezérelt fejlesztés értéke volt. Több, papíron ígéretes ötlet (pl. a sakk-kiterjesztés) a mérésekben nem hozott javulást, míg más, látszólag apró változtatások (zármentes transzpozíciós tábla) jelentős erősödést okoztak. Automatikus verziózás és sok száz játszmás mérések nélkül ezek a döntések csak találgatások lettek volna.

10.2. Továbbfejlesztési lehetőségek

- *Általános tündérbábu-leírás:* jelenleg az Amazon kódba épített bábutípus. A következő lépés egy Betza-jelölésen alapuló, konfigurálható bábuleírás lehetne, amelyből a lépésgenerátor és a támadásszámítás automatikusan felépül, így új bábuk újrafordítás nélkül is felvehetők.

- *Saját NNUE tanítása:* az Amazon értékét a motor jelenleg egy rögzített, +300 centigyalogos korrekcióval becsüli. Önjátszmákon tanított, az Amazont külön bemenetként kezelő háló pontosabb értékelést adhatna.
- *UCI-opciók:* a hash-méret, a szálszám és a változat UCI `setoption` paranccsal is állíthatóvá tehető, és támogatható a `go depth`, `go infinite` és `ponder` mód.
- *Pontosabb erősségmérés:* a Stockfish elleni mérést érdemes lenne a kalibráció időkontrolljával (120 s + 1 s), több szálon és több játszmával megismételni, valamint az Amazon változatban is elvégezni (pl. a Fairy-Stockfish ellen).

Irodalomjegyzék

[1] Ralph Betza: Funny Notation – leaper and rider notation for fairy pieces. The Chess Variant Pages, `https://www.chessvariants.com/piececlopedia.dir/betzanot.html`. Elérve: 2026. október 26.

[2] CCRL – Computer Chess Rating Lists: CCRL Blitz (40/4) rating list. `https://computerchess.org.uk/ccrl/404/`. Elérve: 2026. október 26.

[3] Chess Programming Wiki: Bitboards. `https://www.chessprogramming.org/Bitboards`. Elérve: 2026. október 26.

[4] Chess Programming Wiki: Lazy SMP. `https://www.chessprogramming.org/Lazy_SMP`. Elérve: 2026. október 26.

[5] Chess Programming Wiki: Magic Bitboards. `https://www.chessprogramming.org/Magic_Bitboards`. Elérve: 2026. október 26.

[6] Chess Programming Wiki: Perft Results. `https://www.chessprogramming.org/Perft_Results`. Elérve: 2026. október 26.

[7] Chess Programming Wiki: Pruning, Reductions and Extensions. `https://www.chessprogramming.org/Pruning`. Elérve: 2026. október 26.

[8] Chess Programming Wiki: Static Exchange Evaluation. `https://www.chessprogramming.org/Static_Exchange_Evaluation`. Elérve: 2026. október 26.

[9] Chess Programming Wiki: Transposition Table. `https://www.chessprogramming.org/Transposition_Table`. Elérve: 2026. október 26.

[10] Fabian Fichter: Fairy-Stockfish – chess variant engine. `https://github.com/fairy-stockfish/Fairy-Stockfish`. Elérve: 2026. október 26.

[11] FIDE – Fédération Internationale des Échecs: FIDE Laws of Chess (taking effect from 1 January 2023). `https://handbook.fide.com/chapter/E012023`. Elérve: 2026. október 26.

[12] Donald E. Knuth – Ronald W. Moore: An Analysis of Alpha-Beta Pruning. *Artificial Intelligence*, 6. évf. (1975) 4. sz., 293–326. p.

[13] Stefan Meyer-Kahlen – Rudolf Huber: Description of the Universal Chess Interface (UCI). `https://www.wbec-ridderkerk.nl/html/UCIProtocol.html`. 2004, elérve: 2026. október 26.

[14] Yu Nasu: Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi. Technical report, 2018, Ziosoft Computer Shogi Club. Eredetileg japán nyelven, a 28. World Computer Shogi Championship alkalmából.

[15] David B. Pritchard – John Beasley: *The Classified Encyclopedia of Chess Variants*. 2. kiad. 2007, John Beasley.

[16] Fred Reinfeld: *Win at Chess*. 1958, Dover Publications.

[17] Claude E. Shannon: Programming a Computer for Playing Chess. *Philosophical Magazine*, 41. évf. (1950) 314. sz., 256–275. p.

[18] Daniel Shawul: nnue-probe: Library for probing Stockfish NNUE networks. `https://github.com/dshawul/nnue-probe`. Elérve: 2026. október 26.

[19] The Chess Variant Pages: Amazon (Queen + Knight compound piece). `https://www.chessvariants.com/piececlopedia.dir/amazon.html`. Elérve: 2026. október 26.

[20] The Cute Chess developers: Cute Chess – graphical user interface, command-line interface and library for playing chess. `https://github.com/cutechess/cutechess`. 1.5.1-es verzió, elérve: 2026. október 26.

[21] The Qt Company: Graphics View Framework – Qt 6 documentation. `https://doc.qt.io/qt-6/graphicsview.html`. Elérve: 2026. október 26.

[22] The Stockfish developers: nnue-pytorch: NNUE training code and documentation. `https://github.com/official-stockfish/nnue-pytorch/blob/master/docs/nnue.md`. Elérve: 2026. október 26.

[23] The Stockfish developers: Stockfish – A free and strong UCI chess engine. `https://github.com/official-stockfish/Stockfish`. Elérve: 2026. október 26.

[24] Albert L. Zobrist: A New Hashing Method with Application for Game Playing. 88. Technical Report, 1970, University of Wisconsin, Computer Sciences Department.

Függelék

F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

☐ **Nem használtam** semmilyen generatív MI segédeszközt.
☒ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	**Generatív MI eszköz(ök) neve**	**Érintett részek** (fejezet, oldalszám, hivatkozás)	**Használat becsült aránya** (felhasználási módonként)
A LaTeX-forrás szerkesztése, formázása	Claude (Anthropic), Claude Code	teljes dolgozat	**100**%
Prompt lényegi része	A dolgozat LaTeX-forrásának szerkesztése: ábrák elrendezése és méretezése, tördelés, hivatkozások és képaláírások javítása.		
Adatvizualizáció, grafikonok generálása adatpontok alapján	Claude (Anthropic), Claude Code	8.2. és 9.1. ábra	100%
Prompt lényegi része	Grafikonok és hőtérképek készítése meglévő mérési adatokból: a verziók közötti mérések grafikonja és az Amazon hőtérképei.		